# CHƯƠNG II. XÂY DỰNG CHỈ MỤC ĐẢO CƠ BẢN

Truy xuất thông tin

ThS. Phạm Đức Cường
cuongpd@ptit.edu.vn

Posts and Telecommunications
Institute of Technology

Học viện Công nghệ Bưu chính Viễn thông

# Nội dung

1. Cách xây dựng chỉ mục nghịch đảo cơ bản
2. Chiến lược xây dựng khi bộ nhớ hạn chế
3. Lập chỉ mục phân tán
4. Lập chỉ mục động
5. Nghiên cứu tình huống thực tế

# Nhắc lại việc xây dựng chỉ mục

- Các tài liệu được **phân tích để trích xuất các từ**, và những từ này được lưu lại cùng với **Document ID**

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious

| Term | Doc # |
|---|---|
| I | 1 |
| did | 1 |
| enact | 1 |
| julius | 1 |
| caesar | 1 |
| I | 1 |
| was | 1 |
| killed | 1 |
| i' | 1 |
| the | 1 |
| capitol | 1 |
| brutus | 1 |
| killed | 1 |
| me | 1 |
| so | 2 |
| let | 2 |
| it | 2 |
| be | 2 |
| with | 2 |
| caesar | 2 |
| the | 2 |
| noble | 2 |
| brutus | 2 |
| hath | 2 |
| told | 2 |
| you | 2 |
| caesar | 2 |
| was | 2 |
| ambitious | 2 |

# Bước quan trọng

- Sau khi toàn bộ tài liệu được phân tích, tệp chỉ mục đảo được **sắp xếp theo từng thuật ngữ**

Tập trung vào bước sắp xếp

| Term | Doc # |
|---|---|
| I | 1 |
| did | 1 |
| enact | 1 |
| julius | 1 |
| caesar | 1 |
| I | 1 |
| was | 1 |
| killed | 1 |
| i' | 1 |
| the | 1 |
| capitol | 1 |
| brutus | 1 |
| killed | 1 |
| me | 1 |
| so | 2 |
| let | 2 |
| it | 2 |
| be | 2 |
| with | 2 |
| caesar | 2 |
| the | 2 |
| noble | 2 |
| brutus | 2 |
| hath | 2 |
| told | 2 |
| you | 2 |
| caesar | 2 |
| was | 2 |
| ambitious | 2 |

| Term | Doc # |
|---|---|
| ambitious | 2 |
| be | 2 |
| brutus | 1 |
| brutus | 2 |
| capitol | 1 |
| caesar | 1 |
| caesar | 2 |
| caesar | 2 |
| did | 1 |
| enact | 1 |
| hath | 1 |
| I | 1 |
| I | 1 |
| i' | 1 |
| it | 2 |
| julius | 1 |
| killed | 1 |
| killed | 1 |
| let | 2 |
| me | 1 |
| noble | 2 |
| so | 2 |
| the | 1 |
| the | 2 |
| told | 2 |
| you | 2 |
| was | 1 |
| was | 2 |
| with | 2 |

# RCV1: Tập dữ liệu dùng cho bài giảng này

- Làm ví dụ cho việc áp dụng các **thuật toán xây dựng chỉ mục có khả năng mở rộng**, chúng ta sẽ sử dụng **bộ sưu tập Reuters RCV1**
  - Đây là tập dữ liệu gồm **một năm tin tức của hãng Reuters** (từ 20 tháng 8 năm 1996 đến 19 tháng 8 năm 1997)
  - Bộ dữ liệu trên sklearn đã được **clean**
  - https://github.com/cuongphamduc/RCV1
- Tập dữ liệu này **đủ lớn**, **được công khai** và là **một ví dụ hợp lý** cho mục đích minh họa

# Các thống kê của tập dữ liệu Reuters RCV1

| Ký hiệu | Thống kê | Giá trị (khoảng) |
|---|---|---|
| **N** | Số lượng tài liệu | **800.000** |
| **L** | Số lượng token trung bình mỗi tài liệu | **200** |
| **M** | Số lượng thuật ngữ (từ phân biệt – word types) | **400.000** |
| | Số byte trung bình cho mỗi token (bao gồm khoảng trắng/dấu câu) | **6** |
| | Số byte trung bình cho mỗi token (không gồm khoảng trắng/dấu câu) | **4,5** |
| | Số byte trung bình cho mỗi thuật ngữ | **7,5** |
| | Số lượng posting **không có vị trí** (non-positional postings) | **100.000.000** |

4,5 byte cho mỗi word token vs. 7,5 byte cho mỗi word type: vì sao khác nhau?

# Xây dựng chỉ mục dựa trên sắp xếp

- Khi xây dựng chỉ mục, chúng ta **xử lý từng tài liệu một**
  - Danh sách posting cuối cùng của mỗi thuật ngữ **chưa hoàn chỉnh cho đến khi xử lý xong toàn bộ tập tài liệu**
- Với **8 byte cho mỗi cặp (termID, docID)**, phương pháp này **đòi hỏi rất nhiều bộ nhớ** khi làm việc với các tập dữ liệu lớn
- Trong trường hợp của **RCV1**, **T = 100.000.000** (số lượng posting)
  - Vì vậy, **hiện nay có thể thực hiện hoàn toàn trong bộ nhớ**, nhưng **các collection thực tế thường lớn hơn nhiều**.
  - Ví dụ: **New York Times** cung cấp chỉ mục cho **hơn 150 năm tin tức**.
- **Do đó:** chúng ta **cần lưu trữ các kết quả trung gian trên đĩa** thay vì chỉ dựa vào bộ nhớ chính

# Mở rộng khả năng xây dựng chỉ mục

- Việc xây dựng chỉ mục **hoàn toàn trong bộ nhớ** (*in-memory index construction*) **không thể mở rộng** cho các tập dữ liệu lớn
  - Chúng ta **không thể đưa toàn bộ collection vào bộ nhớ**, sắp xếp, rồi ghi ngược trở lại
- Vậy làm thế nào để **xây dựng chỉ mục cho các tập dữ liệu rất lớn**?
- Câu trả lời cần **xem xét các ràng buộc phần cứng**
  - Bộ nhớ (memory)
  - Đĩa (disk)
  - Tốc độ xử lý (speed)
  - …
- Ôn lại một số kiến thức cơ bản về phần cứng

# Kiến thức cơ bản về phần cứng

- Các máy chủ được sử dụng trong các hệ thống IR hiện nay thường có vài GB bộ nhớ chính, đôi khi lên tới hàng chục GB
- Dung lượng đĩa khả dụng lớn hơn rất nhiều
- Khả năng chịu lỗi rất tốn kém: sử dụng nhiều máy thông thường rẻ hơn nhiều so với việc dùng một máy có khả năng chịu lỗi cao
- Việc truy cập dữ liệu trong **bộ nhớ (RAM)** nhanh hơn rất nhiều so với truy cập dữ liệu trên **đĩa**
- **Tìm kiếm vị trí trên đĩa (disk seek):** Trong lúc đầu đọc của đĩa đang di chuyển để định vị, **không có dữ liệu nào được truyền**
- Vì vậy: Việc truyền **một khối dữ liệu lớn** từ đĩa vào bộ nhớ sẽ **nhanh hơn** so với việc truyền **nhiều khối dữ liệu nhỏ**
- **I/O đĩa dựa trên khối (block-based):** Việc đọc và ghi được thực hiện theo **toàn bộ khối**, chứ không phải các mảnh nhỏ hơn
- **Kích thước khối:** từ **8 KB đến 256 KB**

# Các giả định về phần cứng

| Ký hiệu | Thống kê | Giá trị |
|---|---|---|
| **s** | Thời gian tìm kiếm trung bình của đĩa (average seek time) | **5 ms = 5 × $10^{-3}$ s** |
| **b** | Thời gian truyền cho mỗi byte (transfer time per byte) | **0,02 µs = 2 × $10^{-8}$ s** |
| | Tốc độ xung nhịp của bộ xử lý (processor's clock rate) | **$10^{9}$ $s^{-1}$** |
| **p** | Thời gian cho một phép toán mức thấp (low-level operation) (ví dụ: so sánh & hoán đổi một từ – compare & swap a word) | **0,01 µs = $10^{-8}$ s** |
| | Dung lượng bộ nhớ chính (RAM) | **vài GB** |
| | Dung lượng đĩa | **1 TB hoặc hơn** |

# Sắp xếp bằng cách dùng đĩa như "bộ nhớ"?

- Liệu chúng ta có thể dùng **cùng một thuật toán xây dựng chỉ mục** cho các tập dữ liệu lớn hơn, nhưng **sử dụng đĩa thay cho bộ nhớ** không?
- **Không.** Việc sắp xếp **T = 100.000.000 bản ghi** trực tiếp trên đĩa là **quá chậm** – do **phải thực hiện quá nhiều lần tìm vị trí trên đĩa**
- Vì vậy, chúng ta **cần một thuật toán sắp xếp ngoài (external sorting algorithm)**

# BSBI: Blocked Sort-Based Indexing

- Sắp xếp theo khối để giảm số lần truy cập đĩa
- Các bản ghi có kích thước **8 byte**: *(termID, docID)*
- Những bản ghi này được tạo ra trong quá trình **phân tích tài liệu**
- Bây giờ cần sắp xếp **100 triệu** bản ghi 8 byte này theo **termID**
- Định nghĩa một **khối ≈ 10 triệu** bản ghi
  - Có thể **dễ dàng chứa vài khối trong bộ nhớ**
  - Ban đầu sẽ có **10 khối** như vậy
- Ý tưởng cơ bản của thuật toán:
  - Thu thập các postings cho từng khối
  - Sắp xếp khối đó trong bộ nhớ
  - Ghi khối đã sắp xếp ra đĩa
  - Sau đó **trộn** các khối lại với nhau để tạo thành **một thứ tự đã sắp xếp hoàn chỉnh duy nhất**

# Mã giả BSBI

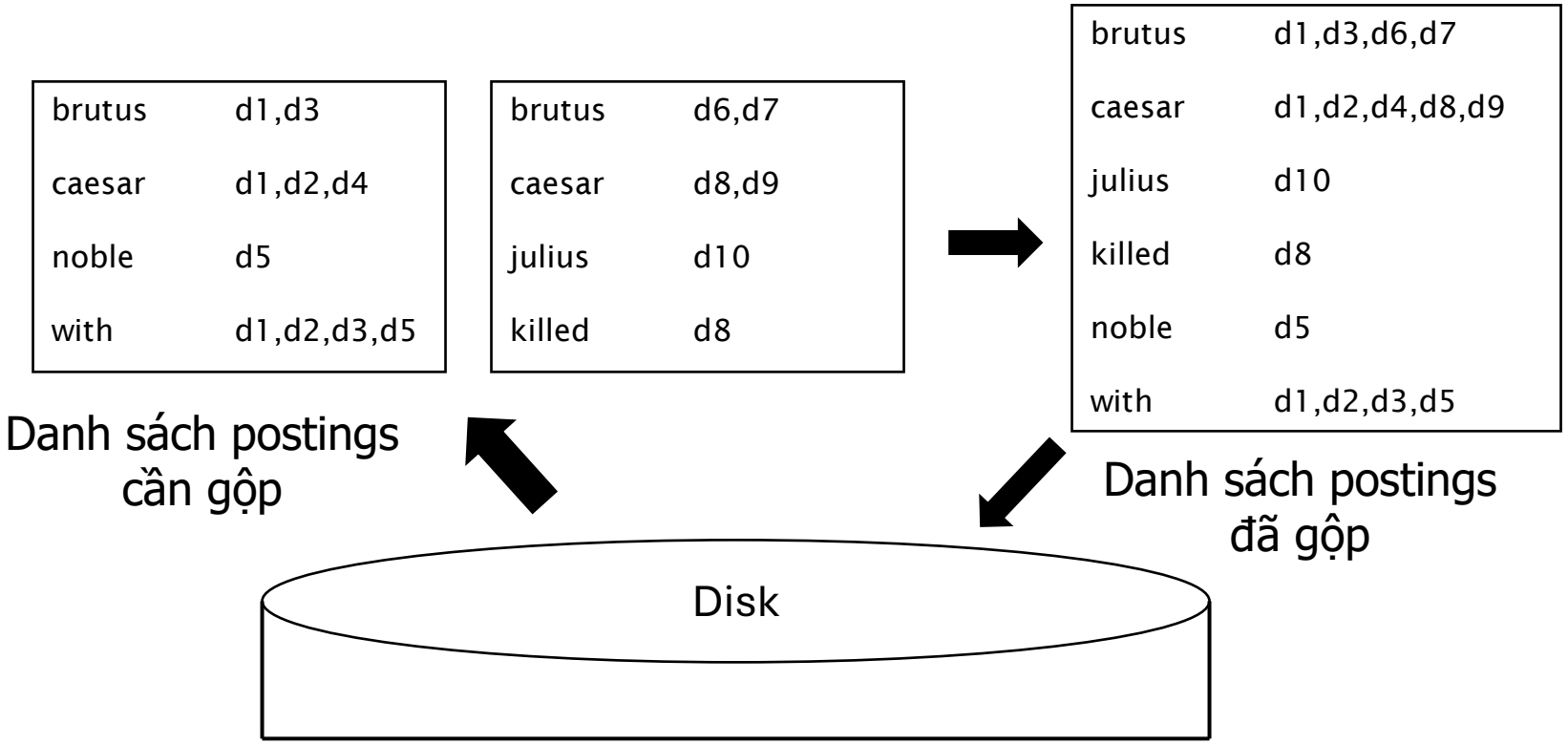
```
BSBIndexConstruction()
n ← 0
while (all documents have not been processed)
do n ← n + 1
    block ← ParseNextBlock()
    BSBI-Invert(block)
    WriteBlockToDisk(block, f_n)
MergeBlocks(f_1, ..., f_n; f_merged)
```

# Sắp xếp 10 khối, mỗi khối gồm 10 triệu bản ghi

- Trước hết, đọc từng khối và sắp xếp bên trong:
  - Quicksort cần số bước kỳ vọng là **O(N log N)**
  - Trong trường hợp hiện tại, **N = 10 triệu**
- Thực hiện 10 lần ước lượng này → thu được **10 dãy đã sắp xếp**, mỗi dãy gồm **10 triệu bản ghi**
- Nếu làm theo cách trực tiếp, cần **2 bản sao dữ liệu trên đĩa**
  - Tuy nhiên, **có thể tối ưu** điều này

# Làm thế nào để trộn các dãy đã sắp xếp?

- Có thể thực hiện **gộp nhị phân**, với một **cây gộp** có **$\log_2 10 = 4$** tầng
- Trong mỗi tầng, **đọc các run vào bộ nhớ theo từng khối 10 triệu, trộn,** rồi **ghi trở lại**



# Làm thế nào để trộn các dãy đã sắp xếp?

- Nhưng hiệu quả hơn là thực hiện **multi-way merge**, trong đó chúng ta **đọc từ tất cả các khối đồng thời**
  - **Mở tất cả các file khối** cùng lúc và duy trì **một bộ đệm đọc (read buffer)** cho mỗi file, và **một bộ đệm ghi (write buffer)** cho file đầu ra
  - Trong mỗi vòng lặp, dùng **hàng đợi ưu tiên (priority queue)** để chọn **termID nhỏ nhất** mà **chưa được xử lý**
  - **Gộp tất cả các postings lists** cho **termID** đó và **ghi ra**
- Miễn là chúng ta **đọc các mẩu đủ lớn (decent-sized chunks)** của mỗi khối vào bộ nhớ và rồi **ghi ra một mẩu đầu ra đủ lớn**, thì chúng ta sẽ **không bị "giết" bởi việc seek của đĩa**

# Vấn đề còn lại của thuật toán dựa trên sắp xếp

- Giả định của chúng ta là: **có thể giữ từ điển trong bộ nhớ**
- Chúng ta cần **từ điển này (tăng dần theo thời gian và không biết trước)** để **cài đặt ánh xạ từ term sang termID**

# SPIMI: Single-pass in-memory indexing

- Lập chỉ mục trong bộ nhớ một lần
- **Ý tưởng chính 1:** Tạo **từ điển riêng cho từng khối – không cần duy trì ánh xạ term–termID giữa các khối**
- **Ý tưởng chính 2: Không sắp xếp**. Thêm các **postings** trực tiếp vào **postings list** ngay khi chúng xuất hiện
- Với hai ý tưởng này, chúng ta có thể **tạo một chỉ mục ngược hoàn chỉnh cho mỗi khối**. Sau đó, các **chỉ mục riêng lẻ này có thể được gộp** lại thành **một chỉ mục lớn duy nhất**.

# SPIMI-Invert

- Việc gộp các khối là tương tự như trong BSBI

```
SPIMI-INVERT(token_stream)
output_file = NEWFILE()
dictionary = NEWHASH()
while (free memory available)
do token ← next(token_stream)
   if term(token) ∉ dictionary
      then postings_list = ADDTODICTIONARY(dictionary, term(token))
      else postings_list = GETPOSTINGSLIST(dictionary, term(token))
   if full(postings_list)
      then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
   ADDTOPOSTINGSLIST(postings_list, docID(token))
sorted_terms ← SORTTERMS(dictionary)
WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
return output_file
```

# SPIMI trong thực tế

| Input token | Dictionary | Sorted dictionary |
| --- | --- | --- |
| Caesar d1 | brutus d1 d3 | brutus d1 d3 |
| with d1 | with d1 d2 d3 d5 | caesar d1 d2 d4 |
| Brutus d1 | noble d5 | noble d5 |
| Caesar d2 | caesar d1 d2 d4 | with d1 d2 d3 d5 |
| with d2 | | |
| Brutus d3 | | |
| with d3 | | |
| Caesar d4 | | |
| noble d5 | | |
| with d5 | | |

# SPIMI: Nén

- Nén giúp **SPIMI** trở nên **hiệu quả hơn nữa**
  - Nén các terms
  - Nén các postings
- Phần này sẽ được **trình bày chi tiết hơn ở phần sau …**

Original publication on SPIMI: Heinz and Zobel (2003)

# Lập chỉ mục phân tán (Distributed indexing)

- Đối với việc lập chỉ mục ở quy mô web
  - Bắt buộc phải sử dụng **cụm máy tính phân tán**
- Các máy riêng lẻ **dễ gặp lỗi**
  - Có thể **chậm đi hoặc bị hỏng một cách không đoán trước**
- Làm thế nào để khai thác hiệu quả một tập hợp các máy như vậy?

# Trung tâm dữ liệu của công cụ tìm kiếm web

- Các trung tâm dữ liệu tìm kiếm web (Google, Bing, Baidu) chủ yếu gồm các **máy phổ thông**
- Các trung tâm dữ liệu này **phân bố trên toàn thế giới**
- Ước tính: **Google có khoảng 2.5 triệu máy chủ** (Gartner, 2016)

# Trung tâm dữ liệu quy mô cực lớn

- Nếu trong một hệ thống **không có khả năng chịu lỗi** với **1000 node**, mỗi node có **99,9% thời gian hoạt động (uptime)**, thì **uptime của toàn bộ hệ thống là bao nhiêu?**
- **Đáp án: 37%** – nghĩa là **63% thời gian có ít nhất một máy chủ bị down**
- **1000 server fail mỗi phút** đối với một hệ thống gồm **1 triệu máy chủ**

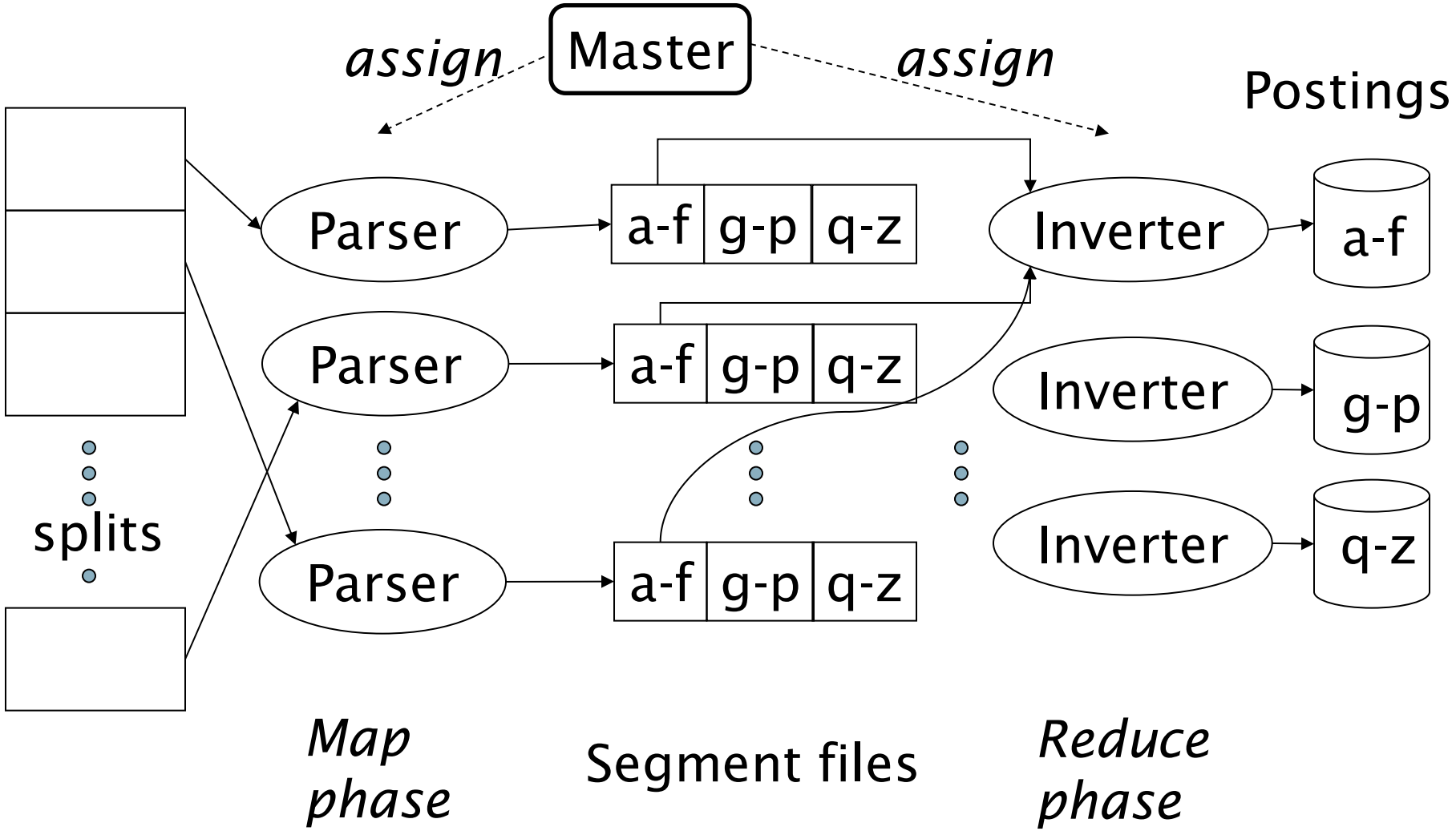
# Lập chỉ mục phân tán

- Duy trì **một máy chủ trung tâm (master machine)** để điều phối công việc lập chỉ mục – được coi là **"an toàn"**
- Chia công việc lập chỉ mục thành **nhiều tập nhiệm vụ có thể chạy song song**
- Máy chủ trung tâm **phân công mỗi nhiệm vụ cho một máy đang rảnh** trong **tập các máy (pool)**

# Các tác vụ song song

- Chúng ta sẽ sử dụng **hai nhóm tác vụ song song**:
  - Phân tích (Parsers)
  - Lập chỉ mục đảo (Inverters)
- Chia tập tài liệu đầu vào thành các **split**
- Mỗi **split** là một **tập con các tài liệu**, tương ứng với **các block trong BSBI/SPIMI**

# Luồng dữ liệu



# Parsers

- Máy chủ trung tâm gán một **split** cho một **máy parser đang rảnh**
- **Parser** đọc **từng tài liệu một** và sinh ra các cặp **(term, doc)**
- Parser ghi các cặp này vào **j phân vùng**
  - Ví dụ: mỗi phân vùng tương ứng với **một khoảng chữ cái đầu của term** (ví dụ: a–f, g–p, q–z) – trong trường hợp này **j = 3**
- Sau đó chuyển sang bước hoàn tất việc lập chỉ mục đảo

# Inverters

- Một **inverter** thu thập **tất cả các cặp (term, doc)** (= **postings**) thuộc về **một phân vùng của term**
- Sau đó **sắp xếp** chúng và **ghi ra các postings list**

# Ví dụ về việc xây dựng chỉ mục

Map:

Caesar

conquered

d1 : C came, C c’ ed.

d2 : C died.

→

<C,d1>, <came,d1>, <C,d1>, <c’ ed, d1>, <C, d2>, <died,d2>

Reduce:

(<C,(d1,d1,d2)>, <died,(d2)>, <came,(d1)>, <c’ ed,(d1)>)

→

(<C,(d1:2,d2:1)><died,(d2:1)>, <came,(d1:1)>,<c’ ed,(d1:1)>)

# Bài tập

- Giả sử rằng mỗi máy trong hệ thống MapReduce có 100 GB dung lượng đĩa. Giả sử thêm rằng danh sách postings của thuật ngữ “the” có kích thước 200 GB. Khi đó, thuật toán MapReduce như đã mô tả sẽ không thể chạy để xây dựng chỉ mục. Bạn sẽ sửa đổi MapReduce như thế nào để nó có thể xử lý được trường hợp này?

# Xây dựng chỉ mục

- Việc xây dựng chỉ mục chỉ là **một giai đoạn**
- Một giai đoạn khác là: **chuyển đổi một chỉ mục được phân vùng theo term** thành **chỉ mục được phân vùng theo tài liệu**
  - **Phân vùng theo term (term-partitioned):** mỗi máy xử lý **một khoảng con các term**
  - **Phân vùng theo tài liệu (document-partitioned):** mỗi máy xử lý **một khoảng con các tài liệu**
- **S**ẽ được thảo luận trong **phần web của học phần**, hầu hết các **công cụ tìm kiếm** đều sử dụng **chỉ mục phân vùng theo tài liệu** vì **cân bằng tải tốt hơn**, v.v

# MapReduce

- Thuật toán xây dựng chỉ mục mà chúng ta vừa mô tả là **một trường hợp cụ thể của MapReduce**
- **MapReduce** (Dean và Ghemawat, 2004) là một **framework mạnh mẽ và đơn giản về mặt khái niệm** cho tính toán phân tán, cho phép thực hiện xử lý phân tán **mà không cần phải tự viết mã cho phần phân phối công việc**
- Họ mô tả **hệ thống lập chỉ mục của Google (khoảng năm 2002)** gồm **nhiều giai đoạn**, và **mỗi giai đoạn đều được cài đặt bằng MapReduce**

# Sơ đồ xây dựng chỉ mục trong MapReduce

- Sơ đồ của các hàm map và reduce
  - **map:** *input* → **list(k, v)**
  - **reduce:** *(k, list(v))* → **output**
- Áp dụng sơ đồ này cho bài toán xây dựng chỉ mục
  - **map:** *tập tài liệu* → **list(termID, docID)**
  - **reduce:** *(<termID$_1$, list(docID)>, <termID$_2$, list(docID)>, …)* → **(postings list$_1$, postings list$_2$, …)**

# Lập chỉ mục động (Dynamic indexing)

- Cho đến nay, chúng ta đã **giả định rằng tập tài liệu là tĩnh**. Nhưng trên thực tế, **hiếm khi đúng như vậy**:
  - **Tài liệu mới** liên tục xuất hiện và **cần được thêm vào**
  - **Tài liệu bị xóa hoặc được chỉnh sửa**
- Điều này có nghĩa là **dictionary và các postings list phải được cập nhật**:
  - **Cập nhật postings** cho các term **đã có trong dictionary**
  - **Thêm các term mới** vào **dictionary**

# Cách tiếp cận đơn giản nhất

- Duy trì một **chỉ mục chính (main index) "lớn"**
- Các **tài liệu mới** đưa vào một **chỉ mục phụ (auxiliary index) "nhỏ"**
- Khi truy vấn, **tìm kiếm trên cả hai chỉ mục** và **gộp kết quả**
- **Xóa tài liệu:**
  - Dùng một **bit-vector vô hiệu hóa (invalidation bit-vector)** cho các tài liệu đã bị xóa
  - Khi trả kết quả truy vấn, **lọc bỏ các tài liệu** dựa trên bit-vector này
  - **Định kỳ**, xây dựng lại (re-index) để **gộp thành một chỉ mục chính duy nhất**

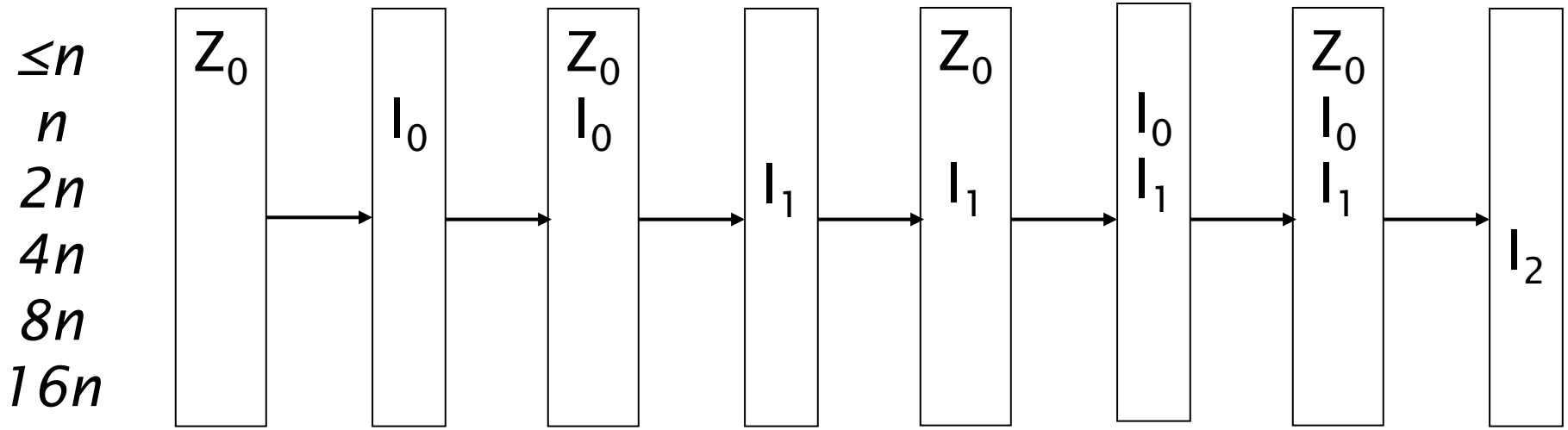
# Các vấn đề với chỉ mục chính và chỉ mục phụ

- **Vấn đề merge thường xuyên** – phải đụng chạm/cập nhật dữ liệu rất nhiều lần
- Hiệu năng kém trong lúc merge
- Thực tế là:
  - Việc **merge chỉ mục phụ vào chỉ mục chính** sẽ **hiệu quả** nếu ta **giữ một file riêng cho mỗi postings list**
  - Khi đó, thao tác merge **chỉ đơn giản là nối thêm**
  - Tuy nhiên cách này đòi hỏi **rất nhiều file** → **không hiệu quả với hệ điều hành**
- Giả định cho phần còn lại của bài giảng
  - Chỉ mục được lưu trong một file lớn duy nhất
- Trong thực tế, người ta dùng **giải pháp trung gian**, ví dụ:
  - Chia nhỏ các postings list **rất lớn**
  - Gom các postings list **độ dài = 1** vào **chung một file**, v.v.

# Gộp theo logarit (Logarithmic merge)

- Duy trì **một dãy các chỉ mục**, trong đó **mỗi chỉ mục có kích thước gấp đôi chỉ mục trước đó**
  - Tại một thời điểm bất kỳ, **chỉ một số lũy thừa của 2** trong dãy này **được tạo**
- Giữ **chỉ mục nhỏ nhất ($Z_0$)** trong **bộ nhớ**
- Các chỉ mục lớn hơn (**$I_0$, $I_1$, …**) được lưu **trên đĩa**
- Khi **$Z_0$ trở nên quá lớn (> n)**
  - Ghi $Z_0$ ra đĩa thành **$I_0$**, **hoặc**
  - Nếu **$I_0$ đã tồn tại**, thì **merge $Z_0$ với $I_0$** để tạo thành **$Z_1$**
- Sau đó:
  - Nếu **chưa có $I_1$** → ghi **$Z_1$** ra đĩa thành **$I_1$**, **hoặc**
  - Nếu **$I_1$ đã tồn tại** → **merge $Z_1$ với $I_1$** để tạo thành **$Z_2$**
- Quá trình này tiếp tục tương tự, tạo ra một cơ chế merge theo kiểu **cộng dồn như phép cộng nhị phân**, giúp **giảm tần suất merge tốn kém** và **cân bằng chi phí cập nhật theo thời gian**

# Gộp theo logarit trong thực tế



# Mã giả gộp theo logarit

```
LMergeAddToken(indexes, Z_0, token)
  Z_0 ← Merge(Z_0, {token})
  if |Z_0| = n
    then for i ← 0 to ∞
           do if I_i ∈ indexes
                then Z_{i+1} ← Merge(I_i, Z_i)
                       (Z_{i+1} is a temporary index on disk.)
                     indexes ← indexes − {I_i}
                else I_i ← Z_i      (Z_i becomes the permanent index I_i.)
                     indexes ← indexes ∪ {I_i}
                     Break
           Z_0 ← ∅
```

```
LogarithmicMerge()
  Z_0 ← ∅      (Z_0 is the in-memory index.)
  indexes ← ∅
  while true
  do LMergeAddToken(indexes, Z_0, getNextToken())
```

# Gộp theo logarit

- Chỉ mục phụ và chỉ mục chính
  - Có **T/n lần merge**, trong đó **T** là **số lượng postings**, và **n** là **kích thước của chỉ mục phụ**
  - **Thời gian xây dựng chỉ mục** là **O($T^2$ / n)** vì trong trường hợp xấu nhất, **mỗi posting bị "đụng tới" T/n lần**
- Gộp theo logarit
  - Mỗi posting chỉ bị merge **tối đa O(log (T/n)) lần**, nên **độ phức tạp là O(T log (T/n))**
  - Do đó, **logarithmic merge hiệu quả hơn nhiều** cho việc **xây dựng chỉ mục**
- Tuy nhiên
  - **Xử lý truy vấn** bây giờ cần **merge O(log (T/n)) chỉ mục**
  - Trong khi đó, nếu chỉ có **một chỉ mục chính và một chỉ mục phụ**, thì chi phí này chỉ là **O(1)**

# Bài tập

- Với n = 2 và 1 ≤ T ≤ 30, hãy thực hiện mô phỏng từng bước của thuật toán Gộp theo logarit. Lập một bảng thể hiện, tại mỗi thời điểm mà T = 2 * k tokens đã được xử lý (1 ≤ k ≤ 15), chỉ số nào trong ba chỉ số $I_0$, …, $I_3$ đang được sử dụng. Ba dòng đầu tiên của bảng được cho như sau:

| | $I_3$ | $I_2$ | $I_1$ | $I_0$ |
|---|---|---|---|---|
| 2 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 1 |
| 6 | 0 | 0 | 1 | 0 |

# Các vấn đề tiếp theo với nhiều chỉ mục

- **Các thống kê trên toàn bộ collection** rất **khó duy trì**
- Ví dụ, khi nói đến **sửa lỗi chính tả (spell-correction)**:
  - Trong số nhiều phương án sửa khác nhau, **ta sẽ hiển thị phương án nào cho người dùng?**
  - Ta có thể muốn **chọn phương án có số lần xuất hiện (hits) nhiều nhất**
- **Làm thế nào để duy trì các lựa chọn hàng đầu** khi tồn tại **nhiều chỉ mục** và **các bit-vector vô hiệu hóa**
- **Một khả năng** là: **Bỏ qua tất cả**, chỉ **dựa vào chỉ mục chính** cho việc sắp thứ tự này
- Chúng ta sẽ còn gặp **nhiều loại thống kê như vậy** được sử dụng trong **xếp hạng kết quả tìm kiếm** ở các phần sau

# Lập chỉ mục động trong các công cụ tìm kiếm

- Tất cả các **công cụ tìm kiếm lớn** hiện nay đều thực hiện **lập chỉ mục động**
- Chỉ mục của họ có **những thay đổi gia tăng rất thường xuyên**
- Ví dụ:
  - Tin tức mới
  - Blog
  - Các trang web theo chủ đề mới, đang “hot”
- Tuy nhiên, **đôi khi** họ cũng **định kỳ xây dựng lại toàn bộ chỉ mục từ đầu**
- Khi đó:
  - **Xử lý truy vấn** sẽ được **chuyển sang chỉ mục mới**
  - Và **chỉ mục cũ sẽ bị xóa**

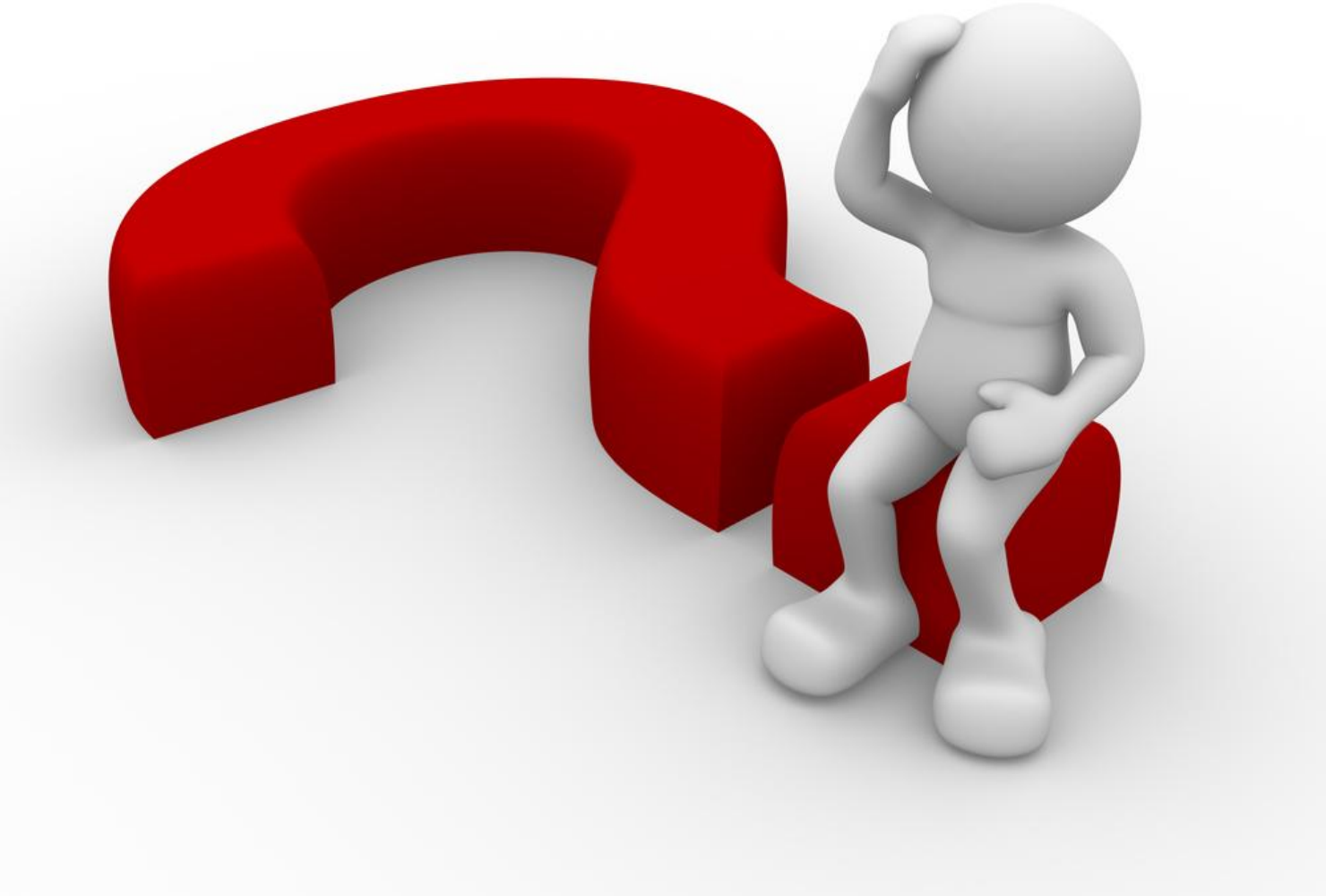
# Earlybird: Tìm kiếm thời gian thực tại X (Twitter)

- Các yêu cầu đối với tìm kiếm thời gian thực:
  - **Độ trễ thấp**, **thông lượng truy vấn cao** khi đánh giá truy vấn
  - **Tốc độ nạp dữ liệu cao** và **dữ liệu sẵn sàng sử dụng ngay lập tức**
  - Đọc và ghi chỉ mục **đồng thời**
  - **Tín hiệu thời gian** chiếm ưu thế

# Earlybird: Tổ chức chỉ mục

- **Earlybird** gồm **nhiều index segments (phân đoạn chỉ mục)**
- Mỗi **phân đoạn** tương đối **nhỏ**, chứa tối đa **$2^{23}$ tweet**
- Mỗi **posting** trong một segment là **một từ 32 bit**:
  - **24 bit** cho **tweet ID**
  - **8 bit** cho **vị trí** trong tweet
- **Tại một thời điểm, chỉ có một segment** được phép **ghi**
  - Segment đủ **nhỏ để nằm hoàn toàn trong bộ nhớ**
  - Các **posting mới** được **nối thêm** trực tiếp vào **postings list**
  - Tuy nhiên, **postings list được duyệt ngược lại** để **ưu tiên các tweet mới hơn**
- Các **segment còn lại** được **tối ưu cho chế độ chỉ đọc**
  - Các **posting được sắp xếp theo thứ tự thời gian ngược** (**tweet mới nhất đứng trước**)

# Các loại chỉ mục khác

- Chỉ mục theo **vị trí**
  - Gặp **vấn đề sắp xếp tương tự** như chỉ mục đảo thông thường…
  - Nhưng **kích thước lớn hơn** (vì phải lưu thêm vị trí xuất hiện)
- Xây dựng **chỉ mục n-gram theo ký tự (Character n-gram indexes)**
  - Khi văn bản được phân tích, **liệt kê tất cả các n-gram**
  - Với mỗi **n-gram**, cần lưu **con trỏ tới tất cả các term trong dictionary chứa n-gram đó** – đây chính là các **"postings"**

# Hỏi đáp